Cheatsheet AZ-104 — 07 · ARM templates and Bicep: structure, deployment, export and conversion

1. Anatomy of an ARM template (JSON)

Section	Required	What it contains
`$schema`	Yes	Schema URL according to the scope: `deploymentTemplate.json#` (resource group), `subscriptionDeploymentTemplate.json#`, `managementGroupDeploymentTemplate.json#`, `tenantDeploymentTemplate.json#`
`contentVersion`	Yes	Template version (`1.0.0.0`); informational only
`parameters`	No	Values requested at deployment: `type` (string, int, bool, object, array, **secureString**, secureObject), `defaultValue`, `allowedValues`, `minLength`, `maxLength`, `minValue`, `maxValue`, `metadata.description`
`variables`	No	Reusable computed values
`functions`	No	User-defined functions (rare)
`resources`	Yes	List of resources: `type`, `apiVersion`, `name`, `location`, `properties`, `dependsOn`, `condition`, `copy`, `tags`, `sku`, `kind`
`outputs`	No	Values returned on completion (IDs, FQDN, keys)

```
{
  "$schema": "https://schema.management.azure.com/schemas/2019-04-01/deploymentTemplate.json#",
  "contentVersion": "1.0.0.0",
  "parameters": {
    "prefix": { "type": "string", "minLength": 3, "maxLength": 11 },
    "location": { "type": "string", "defaultValue": "[resourceGroup().location]" },
    "sku": { "type": "string", "defaultValue": "Standard_LRS", "allowedValues": [ "Standard_LRS", "Standard_GRS" ] }
  },
  "variables": { "stName": "[toLower(concat(parameters('prefix'), uniqueString(resourceGroup().id)))]" },
  "resources": [ {
    "type": "Microsoft.Storage/storageAccounts", "apiVersion": "2023-05-01",
    "name": "[variables('stName')]", "location": "[parameters('location')]",
    "sku": { "name": "[parameters('sku')]" }, "kind": "StorageV2",
    "properties": { "supportsHttpsTrafficOnly": true }
  } ],
  "outputs": { "stId": { "type": "string", "value": "[resourceId('Microsoft.Storage/storageAccounts', variables('stName'))]" } }
}
```

- **Expressions** go between square brackets `[ ]`. Functions that show up: `parameters()`, `variables()`, `resourceGroup().location`, `resourceGroup().id`, `subscription().subscriptionId`, `uniqueString()` (deterministic hash for unique names), `concat()` / `format()`, `toLower()`, `resourceId()`, `reference()` (runtime properties of a resource), `listKeys()` (storage keys), `copyIndex()`, `utcNow()` (only in a parameter's defaultValue), `deployment().name`.
- **dependsOn** sets the order when one resource needs another (NIC → VNet, VM → NIC); `reference()` and `listKeys()` **by name** to a resource in the same template create an implicit dependency; `resourceId()` does not (add `dependsOn`). **copy** deploys N instances (`"copy": { "name": "vms", "count": 3 }` + `copyIndex()`); **condition** deploys only if it is met (`"condition": "[equals(parameters('createPip'), 'yes')]"`).
- Secrets: `secureString` parameter and, in the **parameter file**, a reference to **Key Vault** (`"reference": { "keyVault": { "id": "..." }, "secretName": "adminPassword" }`); the vault needs **"Enabled for template deployment"** and the user needs the `Microsoft.KeyVault/vaults/deploy/action` permission.

2. Bicep: the same, shorter

```
targetScope = 'resourceGroup'                    // subscription | managementGroup | tenant

@minLength(3)
param prefix string
param location string = resourceGroup().location
@allowed([ 'Standard_LRS', 'Standard_GRS' ])
param sku string = 'Standard_LRS'
@secure()
param adminPassword string
var stName = toLower('${prefix}${uniqueString(resourceGroup().id)}')
resource st 'Microsoft.Storage/storageAccounts@2023-05-01' = {
  name: stName
  location: location
  sku: { name: sku }
  kind: 'StorageV2'
  properties: { supportsHttpsTrafficOnly: true }
}

resource vnet 'Microsoft.Network/virtualNetworks@2024-01-01' existing = { name: 'vnet-prod' }   // reference without creating
resource nic 'Microsoft.Network/networkInterfaces@2024-01-01' = [for i in range(0, 3): {
  name: 'nic-${i}'
  location: location
  properties: { ipConfigurations: [ { name: 'ipcfg', properties: { subnet: { id: vnet.properties.subnets[0].id } } } ] }
}]
module web './modules/webapp.bicep' = if (sku == 'Standard_GRS') { name: 'webDeploy', params: { location: location } }

output stId string = st.id
```

- **No explicit dependsOn**: Bicep infers dependencies from the **symbolic names** (`st.id`, `vnet.properties...`). Types and API versions with autocompletion in VS Code (Bicep extension).
- Keywords: `param`, `var`, `resource`, `module` (another .bicep or a template spec / registry), `output`, `existing`, `targetScope`, decorators `@secure()`, `@allowed()`, `@description()`, `@minLength`; loops `[for x in list: {...}]`, conditions `= if (...) {...}`, interpolation `'${a}-${b}'`, `?:` operator.
- Tools: `az bicep install` / `az bicep upgrade` / `az bicep version` · `az bicep build --file main.bicep` (Bicep → JSON) · `az bicep decompile --file template.json` (JSON → Bicep; review the result) · `bicep build` / `bicep decompile` if you have the Bicep CLI. Parameters in `main.bicepparam` (`using 'main.bicep'` + `param prefix = 'contoso'`) or in the classic `.parameters.json`.

3. Deploying

Scope	Azure CLI	PowerShell	Purpose
Resource group	`az deployment group create -g RG1 --template-file main.bicep --parameters main.bicepparam` · `--parameters @main.parameters.json` · `--parameters prefix=contoso`	`New-AzResourceGroupDeployment -ResourceGroupName RG1 -TemplateFile main.bicep -TemplateParameterFile main.parameters.json` · `-prefix contoso`	The usual: resources inside an RG
Subscription	`az deployment sub create -l westeurope --template-file main.bicep`	`New-AzSubscriptionDeployment -Location westeurope -TemplateFile main.bicep` (alias `New-AzDeployment`)	Create RGs, Policy, RBAC at subscription level
Management group	`az deployment mg create -m mg-prod -l westeurope --template-file main.bicep`	`New-AzManagementGroupDeployment -ManagementGroupId mg-prod -Location westeurope ...`	Policy / RBAC for many subscriptions
Tenant	`az deployment tenant create -l westeurope ...`	`New-AzTenantDeployment ...`	Management groups

Option	Detail
Incremental mode (default)	Creates or updates what is in the template; **does not touch** what already exists in the RG and is not listed
Complete mode	Additionally **deletes** the RG resources that are **not** in the template (RG scope only; `--mode Complete` / `-Mode Complete`). "Ensure the RG contains only what the template defines" → Complete. Dangerous: validate first with what-if
What-if	`az deployment group what-if ...` / `--confirm-with-what-if` · `New-AzResourceGroupDeployment -WhatIf` → lists Create / Modify / Delete / NoChange before applying
Validate	`az deployment group validate ...` · `Test-AzResourceGroupDeployment` → syntax and schema, not permissions or quotas
History	Resource group → **Deployments**: inputs, outputs, template, errors; you can **Redeploy**. Maximum **800** per RG (auto-deleted when approaching the limit; a CanNotDelete lock prevents it and deployments fail)
Permissions	Contributor at the scope (`Microsoft.Resources/deployments/write` + the resources); to assign roles or Policy from the template, Owner / UAA / Resource Policy Contributor
Deploying from the portal	"Deploy a custom template" → edit, upload a file or pick a **Quickstart template**; any resource has **Automation options** / "Download a template for automation" before creating it

- The same deployment (same name) can be repeated: it is **idempotent**. Repeated deployment names overwrite the history entry.

4. Export, reuse and organize

What	How	Watch out
Export template of an RG or resource	RG → Export template · resource → Export template · `az group export -n RG1` · `Export-AzResourceGroup -ResourceGroupName RG1 -Path .`	Reflects the **current state**, may miss some property or include hardcoded values: review and parameterize. Does not export secrets
Template of a deployment	RG → Deployments → deployment → **Template**	It is **exactly** the template and parameters that were used
Convert JSON → Bicep	`az bicep decompile --file template.json`	Best from a clean template; warns about what it cannot convert
Template specs	`az ts create --name tsStorage --version 1.0 -g RG1 -l westeurope --template-file main.bicep` · `New-AzTemplateSpec` · deploy: `az deployment group create --template-spec <id>` · `New-AzResourceGroupDeployment -TemplateSpecId <id>`	Template **versioned as an Azure resource**, shared with RBAC (Template Spec Reader) with no repository or SAS
Deployment stacks	`az stack group create --name s1 -g RG1 --template-file main.bicep --action-on-unmanage deleteAll --deny-settings-mode denyDelete`	Manage the lifecycle of what was deployed (delete what leaves the template, deny settings); they replace **Blueprints** (retired)
Nested / linked templates	Nested: template embedded in `Microsoft.Resources/deployments`. Linked: `templateLink.uri` to an **accessible** URL (storage + SAS, GitHub) or `relativePath` with template specs	For modularizing; in Bicep, **modules** are used
Quickstart templates	`azure/azure-quickstart-templates` gallery	Starting point; deployed from the portal with "Deploy to Azure"

5. Common errors and what they mean

Error	Cause	Fix
`InvalidTemplate`	Syntax, malformed expression, parameter without a value	Validate / what-if; check brackets and quotes
`ResourceNotFound` when deploying	Missing `dependsOn` or reference to something that does not exist yet	Add the dependency (in Bicep, use the symbolic name)
`DeploymentQuotaExceeded`	800 deployments in the history	Delete old entries (`az deployment group delete`) or remove the lock that prevents cleaning them up
`RequestDisallowedByPolicy`	An Azure Policy blocks the resource (location, SKU, tag)	Comply with the policy or request an exemption
`SkuNotAvailable` / `OperationNotAllowed` (quota)	SKU not available in the region or quota exhausted	Another region / size or increase the quota
`StorageAccountAlreadyTaken` / `Conflict`	Duplicate global name	`uniqueString()` in the name
`MissingSubscriptionRegistration`	Resource provider not registered	`az provider register --namespace Microsoft.X`
`NoRegisteredProviderFound` / invalid `apiVersion`	Nonexistent `apiVersion` for the type	Use a valid version (`az provider show -n Microsoft.Compute --query "resourceTypes[?resourceType=='virtualMachines'].apiVersions"`)
Resources deleted after deploying	**Complete mode** was used	Use Incremental or include everything in the template
Access denied to the Key Vault secret during deployment	Vault without "enabled for template deployment" or without the deploy/action permission	Enable the option and grant the permission

6. Commands that show up

Task	PowerShell (Az.Resources)	Azure CLI
Deploy to RG	`New-AzResourceGroupDeployment -ResourceGroupName RG1 -TemplateFile main.bicep -TemplateParameterFile main.parameters.json -Mode Incremental`	`az deployment group create -g RG1 --template-file main.bicep --parameters main.bicepparam --mode Incremental`
Inline parameter and URI	`-TemplateUri https://.../azuredeploy.json -TemplateParameterObject @{ prefix = "contoso" }`	`--template-uri https://.../azuredeploy.json --parameters prefix=contoso`
What-if / validate	`New-AzResourceGroupDeployment ... -WhatIf` · `Test-AzResourceGroupDeployment ...`	`az deployment group what-if ...` · `az deployment group validate ...`
Subscription	`New-AzSubscriptionDeployment -Location westeurope -TemplateFile main.bicep`	`az deployment sub create -l westeurope --template-file main.bicep`
History	`Get-AzResourceGroupDeployment -ResourceGroupName RG1` · `Get-AzResourceGroupDeploymentOperation` · `Remove-AzResourceGroupDeployment`	`az deployment group list -g RG1 -o table` · `az deployment operation group list` · `az deployment group delete`
Export	`Export-AzResourceGroup -ResourceGroupName RG1 -Path .` · `Save-AzResourceGroupDeploymentTemplate -ResourceGroupName RG1 -DeploymentName d1`	`az group export -n RG1 > rg.json` · `az deployment group export -g RG1 -n d1`
Bicep	`bicep build main.bicep` · `bicep decompile template.json`	`az bicep build --file main.bicep` · `az bicep decompile --file template.json` · `az bicep upgrade`
Template spec	`New-AzTemplateSpec -Name ts1 -Version 1.0 -ResourceGroupName RG1 -Location westeurope -TemplateFile main.bicep`	`az ts create --name ts1 --version 1.0 -g RG1 -l westeurope --template-file main.bicep`

7. Decision tree

```
What does the wording ask for?
├─ Deploy the same thing repeatedly without changes (idempotent) ..... ARM / Bicep in Incremental mode
├─ Leave the RG exactly as the template ............................... Complete mode (RG only)
├─ See what will change before applying ............................... What-if
├─ Reuse what already exists as a template ............................ Export template (RG / resource) or Deployments → Template
├─ Convert a JSON template to Bicep ................................... az bicep decompile
├─ Share versioned templates with the team ............................ Template specs (RBAC)
├─ Password in the template without exposing it ....................... secureString + Key Vault reference in parameters
├─ Unique name for a storage account .................................. uniqueString(resourceGroup().id)
├─ Three identical VMs ................................................ copy (ARM) / for (Bicep)
├─ Resource only in production ........................................ condition (ARM) / if (Bicep)
├─ Create resource groups or policies from the template ............... Subscription / management group scope deployment
├─ "DeploymentQuotaExceeded" .......................................... Delete history (800) or remove the CanNotDelete lock
└─ Replacement for Blueprints ......................................... Template specs + deployment stacks
```

8. Signals → answer

Wording	Answer
"deploy the template without affecting existing resources"	Incremental (default)
"resources not in the template must be deleted"	Complete
"preview changes before deploying"	`what-if` / `-WhatIf`
"template fails: resource not found"	Missing dependsOn (or reference to a resource not yet created)
"storage account name must be unique", "same template in several RGs"	`uniqueString()`
"password must not be stored in the template or parameter file"	secureString + Key Vault reference (enabled for template deployment)
"deploy N identical VMs"	copy / for loop
"VM deploys only when parameter is true"	condition / if
"reuse the exact template that created the resources"	RG → Deployments → Template
"export template is missing some properties"	Normal: the export reflects the state and not every property; use the deployment's template or complete it
"store templates in Azure and control access with RBAC"	Template specs
"convert Bicep to JSON for a tool that only reads ARM"	`az bicep build`
"deploy to a subscription scope", "create resource groups"	`az deployment sub create` / `New-AzSubscriptionDeployment`
"the deployment history is full"	Delete old deployments (800 limit)
"a resource lock prevents deployments"	CanNotDelete on the RG prevents cleaning up the history
"assign a tag / policy during deployment at subscription"	Subscription-level deployment with `Microsoft.Authorization/policyAssignments`

9. Mistakes to avoid

1. Using Complete mode in a shared RG: it deletes everything not in the template.
2. Forgetting `dependsOn` in JSON (in Bicep, symbolic names resolve it).
3. Putting passwords in clear text: `secureString` + Key Vault.
4. Hardcoding the region: `resourceGroup().location` or a parameter.
5. Expecting `validate` to detect quotas, policies or duplicate names: it only checks schema and syntax; `what-if` goes further.
6. Exporting an RG and deploying it as is without parameterizing names and reviewing properties.
7. Confusing template spec (versioned template) with deployment stack (lifecycle management) or with Blueprints (retired).
8. Deploying at RG level a template whose `$schema` is for subscription (or vice versa).